Assignment 7

Visual Information Processing and Management

Davide Costantini, 856114

11 dicembre 2025

1 CNN Finetuning

Sono stati modificati i seguenti parametri dell'imageDataAugmenter:

- RandXScale e RandYScale: l'immagine viene scalata orizzontalmente e verticalmente di un valore casuale preso all'interno del range [0.5, 1.5]
- RandXShear e RandYShear: viene applicato uno shear orizzontale e verticale di angolo casuale preso all'interno del range [-5 5]
- RandRotation: l'immagine viene ruotata di un angolo casuale preso all'interno del range [-7 7]

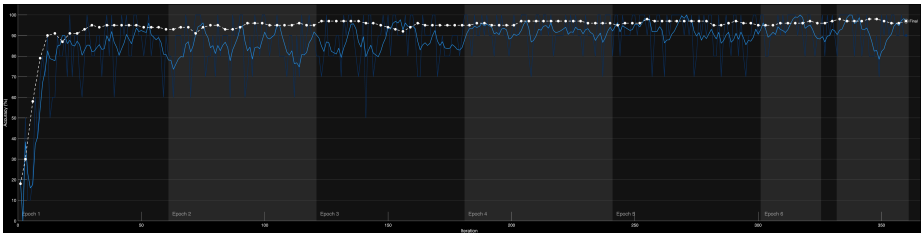


Figura 1: Grafico del training

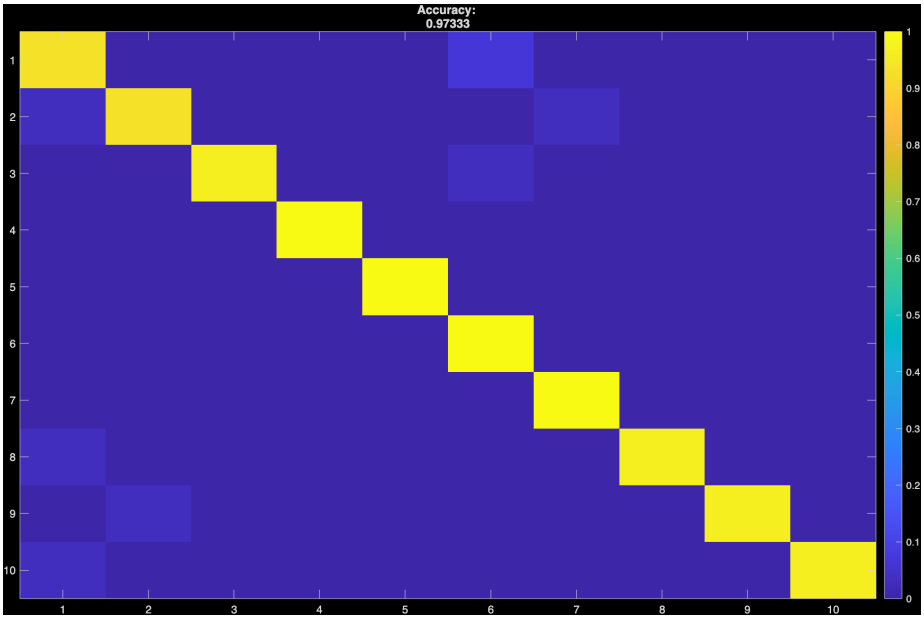


Figura 2: Matrice di confusione

2 Codice

```
net = alexnet;
sz = net.Layers(1).InputSize;
analyzeNetwork(net)

%% Cut layers
layersTransfer = net.Layers(1:end - 3);
layersTransfer = freezeWeights(layersTransfer);

%% Replace layers
numClasses = 10;
layers = [layersTransfer
          fullyConnectedLayer(numClasses, "WeightLearnRateFactor",
    20, "BiasLearnRateFactor", 20);
          softmaxLayer
          classificationLayer];
% analyzeNetwork(net)

%% Preparazione dati
path_db = "../Laboratorio 5/image.orig/"
imds = imageDatastore(path_db);
labels = [];
for ii = 1:size(imds.Files, 1)
    name = imds.Files{ii, 1};
    [p, n, ex] = fileparts(name);
    class = floor(str2double(n) / 100);
    labels = [labels; class];
end
labels = categorical(labels);
imds = imageDatastore(path_db, "Labels", labels);

%% Divisione train-test
[imdsTmp, imdsTest] = splitEachLabel(imds, 0.7, "randomized");
[imdsTrain, imdsValid] = splitEachLabel(imdsTmp, 0.85, "randomized
    ");

%% Data augmentation
translationRange = [-2 2];
scaleRange = [0.5 1.5];
shearRange = [-5 5];
rotationRange = [-7 7];
imageAugmenter = imageDataAugmenter(...
    "RandXReflection", true, ...
    "RandXTranslation", translationRange, ...
    "RandYTranslation", translationRange, ...
    "RandXScale", scaleRange, ...
    "RandYScale", scaleRange, ...
    "RandXShear", shearRange, ...
    "RandYShear", shearRange, ...
    "RandRotation", rotationRange);

augimdsTrain = augmentedImageDatastore(sz(1:2), imdsTrain, "
    DataAugmentation", imageAugmenter);
augimdsValid = augmentedImageDatastore(sz(1:2), imdsValid);
augimdsTest = augmentedImageDatastore(sz(1:2), imdsTest);

%% Configurazione
options = trainingOptions("sgdm", ...
    "MiniBatchSize", 10, ...
    "MaxEpochs", 6, ...
    "InitialLearnRate", 1e-4, ...
```

```
    "Shuffle", "every-epoch", ...
    "ValidationData", augimdsValid, ...
    "ValidationFrequency", 3, ...
    "Verbose", false, ...
    "Plots", "training-progress");

%% Training
netTransfer = trainNetwork(augimdsTrain, layers, options);

%% test
tic
[lab_pred_te, scores] = classify(netTransfer, augimdsTest);
toc

%% Valutazione performance
acc = numel(find(lab_pred_te == imdsTest.Labels)) / numel(
    lab_pred_te)
CM = confusionmat(imdsTest.Labels, lab_pred_te);
CM = CM ./ repmat(sum(CM, 2), 1, size(CM, 2));
accuracy = mean(diag(CM));

figure(1), clf
imagesc(CM), colorbar
title(["Accuracy: " num2str(accuracy)])
```